

A Visual Positioning System for Indoor Blind Navigation

He Zhang and Cang Ye, *Senior Member, IEEE*

Abstract—This paper presents a visual positioning system (VPS) for real-time pose estimation of a robotic navigation aid (RNA) for assistive navigation. The core of the VPS is a new method called depth-enhanced visual-inertial odometry (DVIO) that uses an RGB-D camera and an inertial measurement unit (IMU) to estimate the RNA's pose. The DVIO method extracts the geometric feature (the floor plane) from the camera's depth data and integrates its measurement residuals with that of the visual features and the inertial data in a graph optimization framework for pose estimation. A new measure based on the Sampson error is introduced to describe the measurement residuals of the near-range visual features with a known depth and that of the far-range visual features whose depths are unknown. The measure allows for the incorporation of both types of visual features into graph optimization. The use of the geometric feature and the Sampson error improves pose estimation accuracy and precision. The DVIO method is paired with a particle filter localization (PFL) method to locate the RNA in a 2D floor plan and the information is used to guide a visually impaired person. The PFL reduces the RNA's position and heading error by aligning the camera's depth data with the floor plan map. Together, the DVIO and the PFL allow for accurate pose estimation for wayfinding and 3D mapping for obstacle avoidance. Experimental results demonstrate the usefulness of the RNA in assistive navigation in indoor spaces.

I. INTRODUCTION

According to Lancet Global Health [1], there are about 253 million people with visual impairment, of which 36 million are blind. Since age-related diseases (glaucoma, macular degeneration, diabetes, etc.) are the leading causes of vision loss and the world population is rapidly aging, more people will become blind in the coming decades. Therefore, there is a crucial need to develop navigation aids to assist the blind with their daily mobility need and live independent lives. The problem of independent mobility for a blind person includes wayfinding and obstacle avoidance. Wayfinding is a global problem of planning and following a path towards the destination while obstacle avoidance is a local problem of taking steps without colliding, tripping, or falling. In the literature, a number of Robotic Navigation Aids (RNAs) [2]-[3] have been introduced to assist blind people in wayfinding and/or obstacle avoidance. Among these RNAs, vision-based systems are becoming more popular because the cameras used in these RNAs can provide all of the needed information for navigation, including 6-DOF

device pose (position and orientation) and the surrounding obstacles/objects. The RNA's pose can be used to build a 3D map of the environment, locate the blind traveler in the environment, and guide the traveler to the destination. In the existing literature, monocular camera [4], stereo-cameras [5], [6], RGB-D cameras [7], [8], [9], and 3D time-of-flight (TOF) [10], [11], have been used in these RNAs to perform visual simultaneous localization and mapping (SLAM) to estimate the camera pose for wayfinding. Although the state-of-the-art visual SLAM algorithms [12], [13], [14], [15] have demonstrated their effectiveness in feature-rich environments, their performances are compromised when the operating environment is feature-sparse. Therefore, the previous RNAs cannot work well in a texture-less scene. Moreover, these RNAs require an off-board computer, such as a server [6], [8], [10], [11], a laptop [4], [5], [7], [9], or a tablet computer (e.g., Google Tangle with a quad-core Nvidia Tegra K1 processor [16], [3]) to process the large amount of camera data for navigational decision making. The need for off-board computing resource has hampered the practical use of the RNAs. To address these issues, more robust and computationally efficient methods must be developed for real-time computation on an RNA that has limited computing power. This paper concerns itself with robust and real-time RNA pose estimation for the wayfinding and obstacle avoidance application.

In the robotics community, a visual-inertial system (VINS), consisting of a monocular camera and an inertial measurement unit (IMU), has been popularly used for robust motion estimate. A VINS employs a technique that couples the camera's visual data with the IMU's inertial data to estimate the camera's ego-motion. Such a technique is termed visual-inertial odometry (VIO) in the literature. VIO uses a filtering/smoothing algorithm [17]-[18] to estimate the state vector consisting of the system's pose, velocity, IMU biases, and the depth values of the visual features. The technique requires computing the depth values by structure-from-motion (SFM) for hundreds of visual features so as to initialize these features and re-computing their depth values during the backend optimization process. The computation can be time-consuming and the re-computed depths can be affected by the pose estimation error. Also, many of the state-of-the-art VIO methods [17]-[18] use the *transfer* error to represent the measurement residual for a matched visual-feature-pair between two images and the problem is formulated as one to find an optimal camera pose transformation that minimizes a cost function based on the *transfer* error. In this scheme, the feature point in the 1st image is projected to the 2nd image by perspective projection. The *transfer* error is defined as

This work was supported by the NIBIB and the National Eye Institute of the NIH under award R01EB018117. The content is solely the responsibility of the authors and does not necessarily represent the official views of the funding agencies. H. Zhang and C. Ye are with Computer Science Department, Virginia Commonwealth University, Richmond, VA 23284, USA. (e-mail: hzhang8@vcu.edu, cye@vcu.edu).

the Euclidean distance (aka geometric distance [19]) between the matched feature point and the projected feature point on the second image. This treatment is based on an assumption that the location of the feature point on the 1st image is perfectly accurate, which is not true because image noise, uncalibrated optical distortion, etc. may introduce projective errors. A more accurate error model is the *reprojection* error [19], which takes into account the distance on each of the two images as the result of perspective projection. The Sampson error [20] is the first-order approximation of the reprojection-error-based cost function to be minimized. Compare to the *transfer* error, the Sampson error is more robust and accurate in estimating the homographic [19] or fundamental matrix [21], [22] between two camera frames. However, its performance in the context of a depth-camera-based VIO has not been studied.

In this paper, we propose a so-called depth-enhanced visual-inertial odometry (DVIO) to estimate the pose of an RNA called robocane. The DVIO method uses a sensor package consisting of an RGB-D camera and an IMU for pose estimation. The method improves the state-of-the-art VIO's performance in terms of pose estimation accuracy and computational time by: 1) using the Sampson error to better quantify the measurement residuals for visual features (with a known or unknown depth); 2) using the geometric feature (the floor plane extracted from the camera's depth data) to create additional constraints between the graph's nodes to limit the accumulative pose error; 3) using the depth data from the RGB-D camera for visual feature initialization to avoid the computation incurred by SFM and the need to recompute the visual features' depth values during the course of pose estimation. The computational efficiency allows for real-time implementation of the entire wayfinding software system, consisting of the DVIO and other modules such as Path Planning, Localization, Obstacle Avoidance, etc., on a credit-card-sized board computer. As a result, the system's hardware is compact for installation on a conventional white cane and makes a highly portable RNA possible.

II. RNA PROTOTYPE

As depicted in Fig. 1, the RNA uses an Intel Realsense D435 (RGB-D) Camera and a VN100 IMU (manufactured by VectorNav Technologies, LLC) for pose estimation. The D435 is an RGB-D camera, consisting of a color camera to produce the image data of the scene and an IR stereo camera to generate the corresponding depth data. The resolutions for the color image and depth data are set to 424×240 to produce a 20 fps data stream to the UP Board computer. The D435 is mounted on a white cane with a 25 tilt-up angle to keep the cane's body out of the camera's field-of-view. The length and the weight of the robocane are ~1.55 meters and ~830 grams, respectively. The VN100 is set to output its inertial data at 200 Hz. The robocane uses a mechanism called active rolling tip (ART) [11] to steer the cane to the desired direction of travel (DDT) to guide the blind user. The ART consists of a rolling tip, a gearmotor, a motor drive, and an electromagnetic clutch. A custom control board is built to

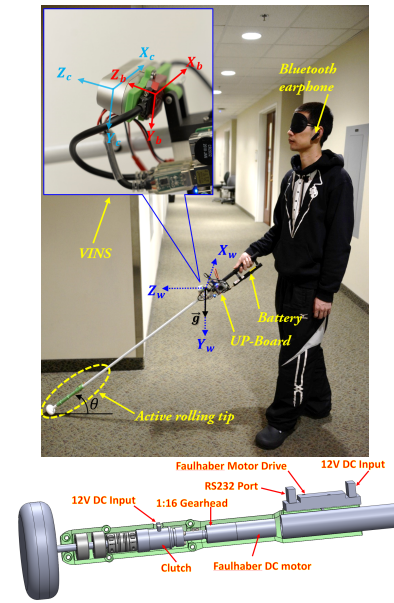


Fig. 1: Top: RNA prototype. The RNA body, camera, and world coordinate systems are denoted by $\{B\}$ (X_b, Y_b, Z_b), $\{C\}$ (X_c, Y_c, Z_c), $\{W\}$ (X_w, Y_w, Z_w), respectively. The initial $\{B\}$ is taken as the world coordinate system $\{W\}$ after performing a rotation to make the Z-axis level and align the Y-axis with the gravity vector $\vec{g}$. θ is the tilt angle between the robocane and the floor. In this paper, the super scripts b and c describe a variable in $\{B\}$ and $\{C\}$, respectively. Bottom: Solidworks drawing for ART

engage and disengage the clutch. When the clutch is engaged, the gearmotor drives the rolling tip and steers the cane to the DDT. When it is disengaged, the rolling tip is disconnected with the gearmotor and the user can swing the robocane just like using a white cane. The UP Board computer control the clutch via its general IO port and the motor drive via the RS-232 port.

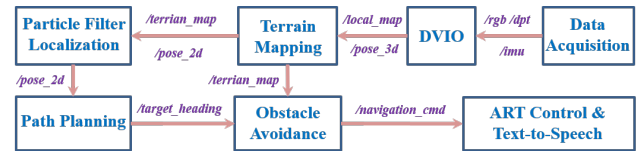


Fig. 2: Software pipeline

III. RNA'S SOFTWARE SYSTEM

The software system of the robocane was developed based on the ROS framework. Each ROS node is an independent functional module that communicates with the other modules through a messaging mechanism. The pipeline of the software is depicted in Fig. 2. The Data Acquisition node acquires and publishes the camera's and the IMU's data, which are subscribed by the DVIO node for real-time pose estimation and 3D map building. The terrain mapping node registers the depth data captured with different camera poses to form a 3D point cloud map, which is used by the DVIO to extract the floor plane for pose estimation. The node also creates a 2D version grid map (out of the 3D point cloud), which is used for obstacle avoidance and localization of the robocane in the 2D floor plan. Based on the robocane's location in the floor plan, the Path Planning module determines the targeted heading to direct the

robocane towards the next Point Of Interest (POI). This information is passed to the Obstacle Avoidance module to compute the Desired Direction of Travel (DDT) that will move the robocane towards the POI without colliding with the surrounding obstacle(s). Based on the DDT, the ART Controller steers the robocane into the DDT, and the Speech Interface sends audio navigation messages to the blind traveler via the Bluetooth headset. Both the tactile and audio information will guide the traveler to move along the planned path. A detailed description of DVIO is given in Section IV. The Particle Filter Localization (PFL), Path Planning, and Obstacle Avoidance modules, as well as the ART Control & Speech Interface, are briefly introduced as follows.

A. Particle Filter Localization (PFL)

The DVIO method accrues pose error over time and a large pose error may cause the wayfinding system to malfunction. To reduce this error, we follow Winterhalter's idea [23] and employ a particle filter (PF) [24] to localize the robocane in a 2D floor plan. Winterhalter's method computes a mobile device's 6-DOF pose in an architectural floor plan by generating a prior 3D map from the architectural drawing and aligning the 3D depth data from the camera with this map. However, the method is slow because it uses a larger number of particles to track the distribution of the 6-DOF pose. In contrast, our method computes the robocane's 3-DOF pose in the 2D floor plan and thus makes real-time computation possible on the UP Board.

B. Path Planning

We use our previous POI-graph method [25] for path planning. In the POI-graph, the graph nodes are the POIs (hallway junctions, elevators, etc.) and each edge between two nodes has a weight that is equal to the distance between the nodes. The A* algorithm is used to find the shortest path from the starting point to the destination. At each POI along the path, the robocane's targeted direction is determined based on the next POI and the current robot position.

C. Obstacle Avoidance

In this work, we employ the Traversability Field Histogram (TFH) [26] method to determine an obstacle-free direction for the robocane. First, a local terrain map surrounding the robocane is converted into a Traversability Map (TM). Then, a Polar Traversability Index (PTI) is computed for each 5° sector of the TM. The smaller the PTI, the more traversable the direction. The PTIs are structured in the form of a histogram. Consecutive sectors with a low PTI form a histogram valley, indicating a walkable direction to the robocane. The valley closest to the robocane's targeted direction is selected and the DDT for the robocane is thus determined. The steering angle for the robocane is calculated based on the DDT and the current robocane heading. The steering angle is then used to control the ART. In addition, a navigational message is generated based on the next POI. This message is conveyed to the user via the speech interface.

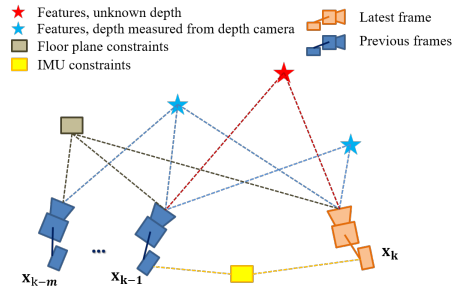


Fig. 3: An example graph of the DVIO method

D. ART Control

To steer the robocane's rolling tip from position A to B and make a heading angle change of $\delta\psi$, the required rotation of the motor is $\delta\mu = CL\delta\psi \cos(\theta)/r$, where C , L , θ , and r are the gearhead reduction ratio, the cane's length, the cane's tilt angle, and the rolling tip's radius, respectively. The initial values for θ (see Fig. 1) and the robocane's roll angle are computed from the 3-axis acceleration data of the IMU. More details are referred to [11].

IV. DVIO

The DVIO method consists of three components: feature tracker, floor detector and state estimator. The feature tracker extracts visual features from an image and tracks them across images while the floor detector extracts the floor plane from the depth data. The state estimator estimates the IMU's state by using the tracked visual features and their depth data, the extracted floor plane, and the IMU measurements. The IMU's state includes its pose, velocity, and biases. The details of the three components are described below.

A. Feature Tracker

The feature tracker detects Harris [19] corner features for each image frame. To attain an affordable computation cost, each image is evenly divided into 8×8 patches. At most 4 features are extracted and tracked for each patch. Therefore, a maximum of 256 feature points is detected for each frame. These features are tracked onto the next image frame by using the KLT [27] method. A fundamental matrix based RANSAC process is devised to remove the outliers and the inliers are passed to the state estimator for pose estimation.

B. Floor Plane Detector

The floor plane is extracted from the first frame and tracked across frames. In this work, the floor plane is described by its normal and the distance from the origin to the plane. At the time the world coordinate system is initialized, the floor plane's normal is $[0, -1, 0]^T$ and the distance can be approximated as $\hat{d}_f^w = L \sin(\theta)$. Data points with a z-coordinate value in $[\hat{d}_f^w - \delta, \hat{d}_f^w + \delta]$ ($\delta = 10$ cm in this work) are used to extract the floor plane. After the extraction, the floor plane will be tracked into the next frame by using the pose estimated by the DVIO at the current frame.

C. State Estimator

State estimation is formulated as a graph optimization problem. Each graph node represents the state vector and the edge between two nodes represents the constraint between them. The state vector is defined as $\mathbf{x}_k = \{\mathbf{x}_{b_k}^w, \mathbf{x}_{b_{k-1}}^w, \dots, \mathbf{x}_{b_{k-m}}^w\}$, where $\mathbf{x}_{b_k}^w = \{\mathbf{t}_{b_k}^w, \mathbf{v}_{b_k}^w, \mathbf{q}_{b_k}^w, \mathbf{b}_a, \mathbf{b}_g\}$ is the IMU's motion state (translation, velocity, rotation, accelerometer bias and gyroscope bias) at the time when the k^{th} image frame is captured, and $m+1$ ($m=3$ in our implementation) is the number of nodes used for graph optimization, i.e., the size of the sliding window for graph optimization is 4 nodes. Similar to Tong's method [28], the visual features with a known depth are used to estimate $\mathbf{x}_k$ in the graph optimization process. In addition, visual features with an unknown depth are also used to create edges in the graph so as to incorporate them into state estimation because they contain the partial pose information (rotation and translation direction of the VINS [29], [30]). Moreover, the extracted floor plane is incorporated into the graph to further reduce the pose estimation error by using our previous method [31]. Fig. 3 shows one example graph used by the DVIO method.

The optimization problem is to find a maximum a posteriori pose estimation that minimizes the sum of the Mahalanobis norms of all measurement residuals given by:

$$\begin{aligned} \mathbf{x}_k^* = \underset{\mathbf{x}_k}{\operatorname{argmin}} & \left(\sum_{(k-1,k) \in C_k} (\mathbf{r}_{k-1,k}^b)^2 + \sum_{(f,k) \in C_p} (\mathbf{r}_{f,k}^p)^2 \right. \\ & \left. + \sum_{(i,k) \in C_{v1}} (\mathbf{r}_{i,k}^v)^2 + \sum_{(i,k) \in C_{v2}} (\mathbf{r}_{i,k}^v)^2 \right) \end{aligned} \quad (1)$$

where $\mathbf{r}_{k-1,k}^b$, $\mathbf{r}_{f,k}^p$ and $\mathbf{r}_{i,k}^v$ are the residual errors related to the IMU, floor plane, and visual feature measurements, respectively. C_p , C_k , C_{v1} , and C_{v2} represent the set of edges for the floor plane, IMU pre-integration, visual features with a known depth, and visual features with an unknown depth, respectively. In this work, $\mathbf{r}_{k-1,k}^b$ and $\mathbf{r}_{f,k}^p$ are computed by using we employ Tong's method [28] and our earlier method [31], respectively, and $\mathbf{r}_{i,k}^v$ computed as the Sampson error [19] (described in the next section) of the related visual feature measurement. We then employ the Ceres solver to solve the optimization problem in Equation (1). Because the D435 camera uses an IR stereovision to measure depth, the measurement error increases quadratically with the true depth. To attain a good pose estimation accuracy, DVIO should only use the depth data of near-range visual features. It has been found through our characterization study that the D435's depth measurement is accurate (error < 2.2 cm) if the true depth is no greater than 2.2 m. Therefore, a near-range (≤ 2.2 m) visual feature is assigned the depth measurement from the D435 and a far-range visual feature is assigned an unknown depth. Different approaches will be introduced to compute the Sampson error for a visual feature with a known depth and that for a visual feature with an unknown depth.

D. Sampson Error

The problem of estimating the camera's pose change between two image frames given a set of matched visual

feature pairs $\mathbf{x}_p \leftrightarrow \mathbf{x}'_p$ for $p = 1, \dots, P$ can be fitted into a common framework with two components [19]:

- a *measurement space* $\mathbb{R}^N$ consisting of measurement vectors $\mathbf{X} = \{\mathbf{x}_p, \mathbf{x}'_p\}$
- a *model* H , representing a subset S of points in $\mathbb{R}^N$. A measurement vector $\mathbf{X}$ that lies inside S is said to satisfy the model. The subspace that satisfies the model is called a *variety* $\mathcal{V}_H$ in $\mathbb{R}^N$

In our case, H is the Euclidean transformation constraint. If a vector $\mathbf{X}$ that lies on $\mathcal{V}_H$, the transformation constraint $\mathcal{C}_H(\mathbf{X}) = 0$ must be satisfied. Here, $\mathcal{C}_H(\mathbf{X})$ is a cost function associated with $\mathbf{X}$. Due to measurement error, $\mathcal{C}_H(\mathbf{X}) \neq 0$ for a real measurement vector. The estimation problem is to find a vector $\hat{\mathbf{X}}$ that is closest to $\mathbf{X}$ and satisfies the model, i.e. $\mathcal{C}_H(\hat{\mathbf{X}}) = 0$. Because of the nonlinearity of $\mathcal{C}_H(\mathbf{X})$, $\hat{\mathbf{X}}$ can only be obtained by an iterative process. $\mathcal{C}_H(\hat{\mathbf{X}})$ can be approximated by a Taylor expansion

$$\mathcal{C}_H(\mathbf{X} + \delta\mathbf{X}) = \mathcal{C}_H(\mathbf{X}) + \frac{\partial \mathcal{C}_H}{\partial \mathbf{X}} \delta\mathbf{X} \quad (2)$$

Here, $\delta\mathbf{X} = \hat{\mathbf{X}} - \mathbf{X}$ and $\hat{\mathbf{X}}$ is desired to lie on the variety to satisfy $\mathcal{C}_H(\hat{\mathbf{X}}) = 0$. Equation (2) can be re-written as

$$\mathbf{J}\delta\mathbf{X} = -\epsilon \quad (3)$$

where $\mathbf{J} = \frac{\partial \mathcal{C}_H}{\partial \mathbf{X}}$ and $\epsilon = \mathcal{C}_H(\mathbf{X}) - \mathcal{C}_H(\mathbf{X} + \delta\mathbf{X})$ is the cost function associated with $\mathbf{X}$. The minimization problem is to find the $\delta\mathbf{X}$ that minimizes $\|\delta\mathbf{X}\|$ subject to Equation (3). The problem can be solved by using Lagrange Multipliers [19] and the solution is given by:

$$\delta\mathbf{X} = -\mathbf{J}^T (\mathbf{J}\mathbf{J}^T)^{-1} \epsilon \quad (4)$$

The Sampson error is defined as:

$$\|\delta\mathbf{X}\|^2 = \epsilon^T (\mathbf{J}\mathbf{J}^T)^{-1} \epsilon \quad (5)$$

1) *Visual Features with a known depth*: Given a visual feature point with a known depth on image frame i , one can compute the Sampson error between image frames i and j by using the geometric distance between the projected point and the measured point in the j^{th} image. However, the Sampson error can become infinite if the depth of the matched visual feature on image j approaches zero (see the Appendix). To avoid this problem, we use the algebraic distance to compute the Sampson error for a visual feature with a known depth. Formally, given a visual feature pair $\mathbf{x}^{c_i} \leftrightarrow \mathbf{x}^{c_j}$ and the inverse depth measurement λ_i , we define the measurement vector in $\mathbb{R}^4$ as $\mathbf{X} = (x_i, y_i, x_j, y_j)^T$ and the variety $\mathcal{V}_H$ by $\mathcal{C}_H(\mathbf{X}) = \mathbf{x}^{c_j} \times \mathbf{x}^{c_i} = \mathbf{0}$, where $\mathbf{x}^{c_j} = (x'_j, y'_j, z'_j)^T = \mathbf{R}_{c_i}^{c_j} \frac{\mathbf{x}^{c_i}}{\lambda_i} + \mathbf{t}_{c_i}^{c_j}$ is the projected visual feature. The cost function is defined by

$$\epsilon = \mathcal{C}_H(\mathbf{X}) = \begin{bmatrix} y'_j - y_j z'_j \\ -x'_j + x_j z'_j \\ y_j x'_j - x_j y'_j \end{bmatrix} \quad (6)$$

Since only two of the three entries in ϵ are linearly independent, the partial derivative of ϵ is a 2×4 matrix, which is computed as

$$\mathbf{J} = \frac{\partial \epsilon}{\partial \mathbf{X}} = \begin{bmatrix} \frac{\partial \epsilon}{\partial x^{c_i}}, \frac{\partial \epsilon}{\partial x^{c_j}} \end{bmatrix} \quad (7)$$

TABLE I: Comparison for the Final End Position Errors (meters)

Dataset	VINS-Mono		DVIO	
	w/o temp. calibration	w temp. calibration	transfor error	Sampson error
1	2.67	1.34	0.83	0.68
2	1.48	1.16	0.29	0.27
3	1.66	0.76	0.41	0.55
4	1.50	0.55	1.04	0.79
5	x	x	0.78	0.73
6	x	1.34	1.26	0.91
7	1.45	0.63	0.74	0.67
8	x	1.21	0.82	0.74
μ, σ	1.75, 0.46	1.00, 0.32	0.77, 0.29	0.67, 0.18

x: failed to converge or EPEN > 3.0m

$$\frac{\partial \epsilon}{\partial \mathbf{x}^{c_i}} = \begin{bmatrix} 0, 1, -y_j \\ -1, 0, x_j \end{bmatrix} \mathbf{R}_{c_i}^{c_j} \begin{bmatrix} \frac{1}{\lambda_i}, 0 \\ 0, \frac{1}{\lambda_i} \\ 0, 0 \end{bmatrix} \quad (8)$$

$$\frac{\partial \epsilon}{\partial \mathbf{x}^{c_j}} = \begin{bmatrix} 0, -z'_j \\ z'_j, 0 \end{bmatrix} \quad (9)$$

After computing $\mathbf{J}$, the Sampson error can be calculated by using Equation (5). Apparently, the accuracy of λ_i plays a critical role in computing the Sampson error. As the D435 camera's depth data become inaccurate when it is beyond 2.2 meters, we use the depth data of a visual feature to compute the Sampson error only if it is no greater than 2.2 meters. Compared with a monocular VIO method [28] that initializes a visual feature's depth at its first observation and updates the depth through graph optimization, DVIO uses the largest inverse depth (i.e., the smallest depth) of the frames within the sliding window for λ_i and the value of λ_i is kept as a constant. These data treatment approaches improve pose estimation accuracy and save computational time.

2) Visual Feature Measurement with an unknown depth:

A pair of matched visual features $\mathbf{x}^{c_i} \leftrightarrow \mathbf{x}^{c_j}$ on the i^{th} and j^{th} image frames satisfy the epipolar geometry between the two frames. The property can be used to compute the Sampson error. As the camera's intrinsic parameters are pre-calibrated, we can use the normalized image coordinates for $\mathbf{x}^{c_i}$ and $\mathbf{x}^{c_j}$, i.e., (x_i, y_i) and (x_j, y_j) , to describe the visual features. The camera's pose change between the two frames is directly related to the Essential matrix $\mathbf{E} = \mathbf{R}_{c_i}^{c_j} \begin{bmatrix} \mathbf{t}_{c_i}^{c_j} \\ 0 \end{bmatrix}_{\times}$, where $[\cdot]_{\times}$ is the skew-symmetric matrix operator. The measurement vector is now defined as $\mathbf{X} = (x_i, y_i, x_j, y_j)^T$ in $\mathbb{R}^4$ and the variety $\mathcal{V}_H$ is defined by $\mathcal{C}_H(\mathbf{X}) = (\mathbf{x}^{c_j})^T \mathbf{E} \mathbf{x}^{c_i} = 0$. In this case, the partial derivative $\mathbf{J}$ has one row, and hence $\mathbf{J}\mathbf{J}^T$ is a scalar. The Sampson error for this visual feature pair is then given by

$$\|\delta_{\mathbf{x}}\|^2 = \frac{((\mathbf{x}^{c_j})^T \mathbf{E} \mathbf{x}^{c_i})^2}{(\mathbf{E} \mathbf{x}^{c_i})_1^2 + (\mathbf{E} \mathbf{x}^{c_i})_2^2 + (\mathbf{E}^T \mathbf{x}^{c_j})_1^2 + (\mathbf{E}^T \mathbf{x}^{c_j})_2^2} \quad (10)$$

where $(\mathbf{E} \mathbf{x})_j^2$ represents the square of the j^{th} entry of vector $\mathbf{E} \mathbf{x}$. In this way, feature points with a depth measurement beyond 2.2 meters are also added to the graph for pose estimation.

V. EXPERIMENTS

A. Performance Evaluation

Similar to our previous work [32], we select VINS-Mono [28] as the benchmark to evaluate the performance of the DVIO. Moreover, the improved version of VINS-Mono [22] that can automatically compensate the mis-synchronization between the camera and IMU data is also compared. To demonstrate the advantage of using the Sampson error, we also implemented the DVIO by using the *transfer* error based cost function for each visual feature. For a fair comparison, both VINS-Mono and DVIO uses a 4-frame sliding window. Eight datasets were collected by holding the robocane and walking along a 20-meter straight line at a speed of ~ 0.7 m/s. The ground truth position of the endpoint is $[0, 0, 20]$. We use the end-point-error-norm (EPEN) to evaluate the accuracy of the pose estimation methods. The results are tabulated in Table I, from which we can see that: 1) DVIO achieves a better pose estimation accuracy than VINS-Mono; 2) the use of the Sampson error improves both the accuracy and robustness of DVIO. The run-time of the Sampson-Error-based DVIO on the Up Board is about 57ms, making real-time implementation (~ 18 fps) of the VPS possible.

B. Evaluation of the Visual Positioning System (VPS)

We tested the VPS by walking with the robocane in a number of looped trajectories in the Engineering East Building of our university. The results demonstrated that the VPS functioned very well in locating itself in the 2D floor plan. Fig. 4 shows the robocane's trajectory (~ 180 meters) estimated by the VPS by using the DVIO and PFL methods for one of the experiments. It can be seen that the system performed well in locating the device in the floor plan. The end point error norm of the test in Fig. 4 is about 0.42 meters.

C. Human Subject Experiments

We recruited five sighted human subjects (three males and two females) outside the research team to test the usefulness of the robocane in assistive navigation. Each subject was blindfolded to perform three navigation tasks in the Engineering East Building. These tasks started at the same starting point RM 2264 and ended at three different destinations, RM 2252, RM 2232, and the elevator. For each task, the human subjects first performed the experiment twice by using the robocane. They were allowed to memorize the location and the path towards the destination during the experiment. Then, they repeated the same task twice by using a conventional white cane instead. They stopped whenever they thought the destination had been reached. The path-lengths for tasks I (destination: RM 2252), II (destination: RM 2232) and III (destination: the elevator) are about 35, 45, and 80 meters, respectively. Obstacles were placed along the paths (see Fig. 5) to test both the wayfinding and obstacle avoidance functions of the robocane. The EPEN and the Number of Successful Trials (NST) are used as performance indexes for localization accuracy. During each test, the number of times the robocane hit an obstacle (NHO), the number of times it hit a wall (NHW), the EPEN, and the time used are recorded.

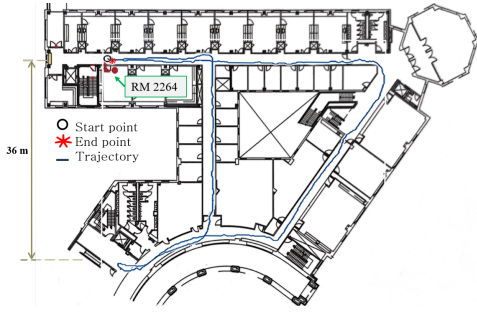
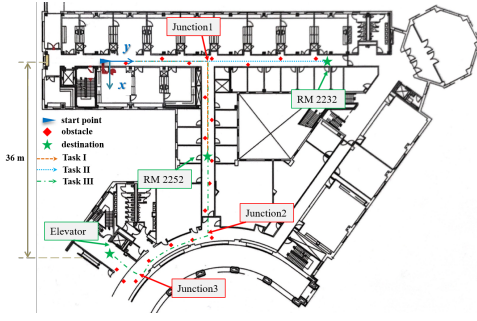


Fig. 4: Floor Plan of East Hall with reference trajectory



(a) Locations of obstacles and task destinations in the floor plan



(b) Snapshots of the scenes at start point and Junctions 1,2

Fig. 5: Experimental Settings for Human Subject Tests

When the EPEN is less than 1.5 meters, the test is taken as a successful one. The Number of Successful Trials (NST) is recorded for each subject. The results for tasks I, II, and III are tabulated in Table II, III, and IV, respectively. It can be seen that with the assistance of the robocane, the success rates of arriving at the destinations are 100%, 100%, and 90% with an average EPEN of 0.25, 0.12 and 0.67 meters, respectively. In contrast, the success rates when using the white cane are 30%, 30%, and 30% with an average EPEN of 0.3, 0.63, and 0.75 meters, respectively. Also, with the robocane, the averaged NHO and NHW for the tests using the robocane are much smaller than that of the cases using the white cane for all three tasks. The results demonstrate that the robocane is effective in assistive navigation (wayfinding and obstacle avoidance) in an indoor environment.

VI. CONCLUSIONS

A new VPS is introduced for an RNA to assist the visually impaired with wayfinding and obstacle avoidance. The VPS is developed based on a new DVIO method that integrates the geometric feature and the visual features of the scene as well as the inertial data for accurate estimation of the 6-DOF RNA pose by graph optimization. The Sampson error is used to quantify the residual errors of visual feature measurements (with and w/o a known depth) and allow

the incorporation of both types of visual features into the graph optimization process. The use of the geometric feature and the Sampson error results in a better pose estimation performance. In addition, numerous treatments are taking to allow for real-time implementation of the DVIO method on the UP Board computer. To serve the purpose of navigation in a large indoor space, the DVIO-estimated RNA position and heading is refined by the PFL method by aligning the camera depth data with the 2D floor plan map. Experimental results demonstrate that: 1) the DVIO method has a better pose estimation performance (in terms of accuracy and repeatability) than the state-of-the-art VIO method; 2) the use of the Sampson error results in a better pose estimation performance; 3) the VPS, based on the DVIO and the PFL methods, is effective in assistive wayfinding and obstacle avoidance.

TABLE II: Result of Task I (Destination: RM 2252)

Subject	With the RNA				With White Cane			
	NST	EPEN	NHO	NHW	NST	EPEN	NHO	NHW
1	2/2	0.55	0.5	0.0	0/2	1.5	x	x
2	2/2	0.25	0.0	0.0	1/2	0.0	4	3
3	2/2	0.15	0.5	0.5	0/2	1.5	x	x
4	2/2	0.05	0.5	1.0	1/2	1.0	1	5
5	2/2	0.25	0.0	1.0	1/2	0.5	5	6
Avg.	100%	0.25	0.3	0.5	30%	0.9	2	2.8

TABLE III: Result of Task II (Destination: RM 2232)

Subject	With the RNA				With White Cane			
	NST	EPEN	NHO	NHW	NST	EPEN	NHO	NHW
1	2/2	0.05	0.0	1.0	2/2	1.15	7.5	16
2	2/2	0.05	0.0	0.5	0/2	1.5	x	x
3	2/2	0.0	1.0	0.0	0/2	1.5	x	x
4	2/2	0.05	1.5	0.5	1/2	0.1	6	1
5	2/2	0.45	0.5	0.5	0/2	1.5	x	x
Avg.	100%	0.12	0.6	0.5	30%	1.15	2.7	8.5

TABLE IV: Result of Task III (Destination: elevator)

Subject	With the RNA				With White Cane			
	NST	EPEN	NHO	NHW	NST	EPEN	NHO	NHW
1	2/2	0.1	2.5	2.0	0/2	1.5	x	x
2	2/2	0.75	1.5	1.0	0/2	1.5	x	x
3	2/2	1.0	1.5	0.5	0/2	1.5	8	7
4	1/2	1.5	2.0	1.0	2/2	0.0	7	5.5
5	2/2	0.0	1.0	2.0	0/2	1.5	x	x
Avg.	90%	0.67	1.7	1.3	30%	1.2	7.5	6.25

x: failed to arrive destination due to missing a turning point or EPEN > 1.5m

APPENDIX

Sampson Error based on Geometric Distance: Given a feature-pair $(\mathbf{x}^{c_i}, \mathbf{x}^{c_j})$ and the inverse depth measurement λ_i , we define the measurement vector as $\mathbf{X} = (x_i, y_i, x_j, y_j)^T$ in $\mathbb{R}^4$. We also define the variety $\mathcal{V}_H$ by $\mathcal{C}_H(\mathbf{X}) : \hat{\mathbf{x}}^{c_j} - \mathbf{x}^{c_j} = \mathbf{0}$, where $\hat{\mathbf{x}}^{c_j} = \mathbf{x}^{c_j} / z_j$ and $\mathbf{x}^{c_j} = (x_j, y_j, z_j)^T = \mathbf{R}_{c_i}^{c_j} \frac{\mathbf{x}^{c_i}}{\lambda_i} + \mathbf{t}_{c_i}^{c_j}$. The derivative matrix is computed as:

$$\frac{\partial \epsilon}{\partial \mathbf{x}^{c_i}} = \begin{bmatrix} \frac{1}{z_j'}, 0, \frac{-x_j'}{(z_j')^2} \\ 0, \frac{1}{z_j'}, \frac{-y_j'}{(z_j')^2} \end{bmatrix} \mathbf{R}_{c_i}^{c_j} \begin{bmatrix} \frac{1}{\lambda_i}, 0 \\ 0, \frac{1}{\lambda_i} \\ 0, 0 \end{bmatrix} \quad (11)$$

$$\frac{\partial \epsilon}{\partial \mathbf{x}^{c_j}} = \begin{bmatrix} -1, 0 \\ 0, -1 \end{bmatrix} \quad (12)$$

When z_j' is close to zero, $\frac{\partial \epsilon}{\partial \mathbf{x}^{c_i}}$ approaches to infinity. This may happen if the camera is moving towards a feature point.

REFERENCES

- [1] R. R. Bourne, S. R. Flaxman, T. Braithwaite, M. V. Cicinelli, A. Das, J. B. Jonas, J. Keeffe, J. H. Kempen, J. Leasher, H. Limburg *et al.*, "Magnitude, temporal trends, and projections of the global prevalence of blindness and distance and near vision impairment: a systematic review and meta-analysis," *The Lancet Global Health*, vol. 5, no. 9, pp. e888–e897, 2017.
- [2] J. A. Hesch and S. I. Roumeliotis, "Design and analysis of a portable indoor localization aid for the visually impaired," *The International Journal of Robotics Research*, vol. 29, no. 11, pp. 1400–1415, 2010.
- [3] B. Li, J. P. Muñoz, X. Rong, Q. Chen, J. Xiao, Y. Tian, A. Ardit, and M. Yousuf, "Vision-based mobile indoor assistive navigation aid for blind people," *IEEE transactions on mobile computing*, vol. 18, no. 3, pp. 702–714, 2018.
- [4] S. Treuillet, E. Royer, T. Chateau, M. Dhome, J.-M. Lavest *et al.*, "Body mounted vision system for visually impaired outdoor and indoor wayfinding assistance," in *CVHI*, 2007.
- [5] J. M. Saez, F. Escolano, and A. Penalver, "First steps towards stereo-based 6dof slam for the visually impaired," in *IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)-Workshops*, 2005, pp. 23–23.
- [6] V. Pradeep, G. Medioni, and J. Weiland, "Robot vision for the visually impaired," in *IEEE Computer Society Conference on Computer Vision and Pattern Recognition-Workshops*, 2010, pp. 15–22.
- [7] B. Li, X. Zhang, J. P. Muñoz, J. Xiao, X. Rong, and Y. Tian, "Assisting blind people to avoid obstacles: An wearable obstacle stereo feedback system based on 3d detection," in *IEEE International Conference on Robotics and Biomimetics (ROBIO)*. IEEE, 2015, pp. 2307–2311.
- [8] Y. H. Lee and G. Medioni, "Rgb-d camera based navigation for the visually impaired," in *Proceedings of the Robotics: Science and Systems*, 2011.
- [9] H. Takizawa, S. Yamaguchi, M. Aoyagi, N. Ezaki, and S. Mizuno, "Kinect cane: An assistive system for the visually impaired based on three-dimensional object recognition," in *IEEE/SICE international symposium on system integration (SII)*, 2012, pp. 740–745.
- [10] H. Zhang and C. Ye, "An indoor navigation aid for the visually impaired," in *IEEE International Conference on Robotics and Biomimetics (ROBIO)*, 2016, pp. 467–472.
- [11] C. Ye, S. Hong, X. Qian, and W. Wu, "Co-robotic cane: A new robotic navigation aid for the visually impaired," *IEEE Systems, Man, and Cybernetics Magazine*, vol. 2, no. 2, pp. 33–42, 2016.
- [12] J. Engel, T. Schöps, and D. Cremers, "Lsd-slam: Large-scale direct monocular slam," in *European conference on computer vision*. Springer, 2014, pp. 834–849.
- [13] F. Endres, J. Hess, N. Engelhard, J. Sturm, D. Cremers, and W. Burgard, "An evaluation of the rgb-d slam system," in *IEEE International Conference on Robotics and Automation*, vol. 3, no. c, 2012, pp. 1691–1696.
- [14] J. Engel, V. Koltun, and D. Cremers, "Direct sparse odometry," *IEEE transactions on pattern analysis and machine intelligence*, vol. 40, no. 3, pp. 611–625, 2017.
- [15] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardos, "Orb-slam: a versatile and accurate monocular slam system," *IEEE transactions on robotics*, vol. 31, no. 5, pp. 1147–1163, 2015.
- [16] R. Jafri, R. L. Campos, S. A. Ali, and H. R. Arabnia, "Visual and infrared sensor data-based obstacle detection for the visually impaired using the google project tango tablet development kit and the unity engine," *IEEE Access*, vol. 6, pp. 443–454, 2017.
- [17] A. I. Mourikis and S. I. Roumeliotis, "A multi-state constraint kalman filter for vision-aided inertial navigation," in *Proceedings IEEE International Conference on Robotics and Automation*, 2007, pp. 3565–3572.
- [18] T. Qin and S. Shen, "Online temporal calibration for monocular visual-inertial systems," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2018, pp. 3662–3669.
- [19] R. Hartley and A. Zisserman, *Multiple view geometry in computer vision*. Cambridge university press, 2003.
- [20] P. D. Sampson, "Fitting conic sections to "very scattered" data: An iterative refinement of the bookstein algorithm," *Computer graphics and image processing*, vol. 18, no. 1, pp. 97–108, 1982.
- [21] Y. Dai, H. Li, and L. Kneip, "Rolling shutter camera relative pose: Generalized epipolar geometry," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 4132–4140.
- [22] M. Brückner, F. Bajramovic, and J. Denzler, "Experimental evaluation of relative pose estimation algorithms," in *International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications*, 2008, pp. 431–438.
- [23] W. Winterhalter, F. Fleckenstein, B. Steder, L. Spinello, and W. Burgard, "Accurate indoor localization for rgb-d smartphones and tablets given 2d floor plans," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2015, pp. 3138–3143.
- [24] G. Grisetti, C. Stachniss, W. Burgard *et al.*, "Improved techniques for grid mapping with rao-blackwellized particle filters," *IEEE transactions on Robotics*, vol. 23, no. 1, p. 34, 2007.
- [25] H. Zhang and C. Ye, "An indoor wayfinding system based on geometric features aided graph slam for the visually impaired," *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, vol. 25, no. 9, pp. 1592–1604, 2017.
- [26] C. Ye, "Navigating a mobile robot by a traversability field histogram," *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 37, no. 2, pp. 361–372, 2007.
- [27] B. D. Lucas, T. Kanade *et al.*, "An iterative image registration technique with an application to stereo vision," 1981.
- [28] T. Qin, P. Li, and S. Shen, "Vins-mono: A robust and versatile monocular visual-inertial state estimator," *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.
- [29] J.-P. Tardif, Y. Pavlidis, and K. Daniilidis, "Monocular visual odometry in urban environments using an omnidirectional camera," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2008, pp. 2531–2538.
- [30] J. Zhang, M. Kaess, and S. Singh, "Real-time depth enhanced monocular odometry," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2014, pp. 4973–4980.
- [31] H. Zhang and C. Ye, "Plane-aided visual-inertial odometry for pose estimation of a 3d camera based in-door blind navigation," in *28th British Machine Vision Conference*, 2017.
- [32] H. Zhang, L. Jin, H. Zhang, and C. Ye, "A comparative analysis of visual-inertial slam for assisted wayfinding of the visually impaired," in *IEEE Winter Conference on Applications of Computer Vision*, 2019, pp. 210–217.